浮点三角函数计算（方案 Q）：单输出、单流水线完整推导

> 本文档采用 **Quadrant 归约方案（Q）**：
>
> - 参数归约使用 $\frac{2}{\pi}$
> - 取整方式为 $\lfloor \cdot \rfloor$（floor）
> - 仅构建一条函数流水线
> - 函数核一次同时给出 $\sin(\frac{\pi}{2}y)$ 与 $\cos(\frac{\pi}{2}y)$
> - 硬件每次只输出一个目标函数（`sin` 或 `cos`）

---

1. 问题定义

输入： 任意浮点数 x，以及目标选择信号 `mode`（`SIN` / `COS`）

输出： 仅输出一个函数值：

- `mode = SIN`：输出 $\sin(x)$
- `mode = COS`：输出 $\cos(x)$

输出为浮点格式，满足 faithful rounding（忠实舍入）。

1.1 浮点数格式

$$x = (-1)^{S_x} \times 1.F_x \times 2^{E_x - E_0}$$

字段	位宽	含义
exn_x	2	异常编码（零、无穷、NaN）
S_x	1	符号位
E_x	w_E	指数（存储值）
F_x	w_F	尾数小数部分

其中 $E_0 = 2^{w_E-1} - 1$ 为指数偏移。

---

2. 总体架构（单流水线）

```
输入 x (浮点), mode
        │
        ▼
┌──────────────────────┐
│ 参数归约 (Q方案)      │  → 输出: q = k mod 4, y∈[0,1), Sx
│ p = |x|·(2/π)         │
│ k = floor(p), y = p-k │
└──────────┬───────────┘
                │
                ▼
┌──────────────────────┐
│ 双输出函数核          │  → 同时计算
│ (HOTBM + 乘法)        │     s(y)=sin(πy/2), c(y)=cos(πy/2)
└──────────┬───────────┘
                │
                ▼
┌──────────────────────┐
│ 选择/符号重构         │  → 输出目标函数值
│ (由 mode, q, Sx 决定) │
└──────────┬───────────┘
                │
                ▼
         result (浮点)
```

关键点：保持单流水线，但函数核内部直接提供两种基函数：

$$s(y) = \sin\left(\frac{\pi}{2}y\right), \quad c(y) = \cos\left(\frac{\pi}{2}y\right), \quad y \in [0, 1).$$

---

3. 参数归约：方案 Q 的数学定义

3.1 基本归约

对输入先取绝对值（符号在最后重构）：

$$u = |x|.$$

定义

$$p = u \cdot \frac{2}{\pi}, \qquad k = \lfloor p \rfloor, \qquad y = p - k \in [0, 1).$$

于是

$$u = (k + y)\,\frac{\pi}{2} = k\frac{\pi}{2} + y\frac{\pi}{2}.$$

令

$$q = k \bmod 4,$$

只需 q 的 2 位即可决定象限映射。

3.2 与目标函数的统一映射

定义两个基函数（同一个 y 输入、同一拍产出）：

$$s(y) = \sin\left(\frac{\pi}{2}y\right), \qquad c(y) = \cos\left(\frac{\pi}{2}y\right).$$

则对 $u = |x|$，$\sin(u)$ 与 $\cos(u)$ 都可由 $\{\pm s(y), \pm c(y)\}$ 组合得到。

这意味着不再需要补角恒等式

$$\cos\left(\frac{\pi}{2}y\right) = \sin\left(\frac{\pi}{2}(1 - y)\right)$$

去做参数变换，因此可移除 `t=1-y` 的 pre-map 级。

3.3 为什么这里选择 $\frac{2}{\pi}$ + floor

在“**单输出、单流水线**”约束下，设计目标是：

1. 保持一条流水线
2. 只保留一种表结构
3. 归约与重构逻辑尽量简单

采用 $\frac{2}{\pi}$ + floor 的好处：

- 归约直接得到 $y \in [0, 1)$（天然适配 $s(y), c(y)$）
- 取整为 floor，可由截断整数部分完成
- `SIN` / `COS` 差异仅体现在“选择 $s(y)$ 还是 $c(y)$”和最终符号

---

4. 参数归约：硬件实现细节（Payne–Hanek）

4.1 为什么不能直接用普通浮点乘法

当 u 很大时，$u \cdot \frac{2}{\pi}$ 的低位（决定 $k \bmod 4$ 与小数 y）非常敏感。

若仅用固定精度常量 + 普通浮点乘法，乘积 ULP 可能远大于 1，导致：

- 象限位 q 误判
- 小数 y 失真

因此仍需 Payne–Hanek：只提取“对 q 与 y 有效”的窗口。

4.2 Payne–Hanek 有效窗口：从“能算”到“算对”

下面给出方案 Q（$2/\pi$ + floor）下可直接落 RTL 的窗口推导。

4.2.1 部分积位置公式

设

$$u = 1.F_x \cdot 2^E, \qquad E = E_x - E_0,$$

并把常量与尾数展开为

$$\frac{2}{\pi} = \sum_i c_i 2^i, \qquad 1.F_x = \sum_{j=0}^{w_F} f_j 2^{-j}.$$

则乘积中每个部分积落位满足

$$\text{bitpos} = i + E - j.$$

对固定 c_i，它影响结果位区间是

$$[i + E - w_F,\ i + E].$$

4.2.2 方案 Q 真正需要哪些位

Q 方案定义

$$p = u \cdot \frac{2}{\pi} = k + y, \quad k = \lfloor p \rfloor, \quad q = k \bmod 4, \quad y \in [0, 1).$$

因此只需：

- 整数位 bit1..bit0（得到 q）
- 小数位到 bit$-(w_F + g + g_K)$（保证后续核与最终舍入）

注意这里是 **floor**，不需要“就近舍入”额外判定位；这点与 Octant/round-to-nearest 不同。

4.2.3 左截断（高位）条件

若某个 c_i 的最低影响位都高于 bit1，则可丢弃。条件为

$$i + E - w_F > 1 \Longrightarrow i > w_F - E + 1.$$

所以保留上界是

$$i \le i_{\max} = w_F - E + 1.$$

4.2.4 右截断（低位）条件

若某个 c_i 的最高影响位都低于目标最低位 bit$-(w_F + g + g_K)$，也可丢弃。条件为

$$i + E < -(w_F + g + g_K) \Longrightarrow i < -(E + w_F + g + g_K).$$

所以保留下界是

$$i \geq i_{\min} = -(E + w_F + g + g_K).$$

4.2.5 有效窗口大小（常数宽度）

窗口为

$$i \in [i_{\min}, i_{\max}] = [-(E + w_F + g + g_K),\ w_F - E + 1].$$

宽度

$$\begin{aligned} W &= i_{\max} - i_{\min} + 1 \\ &= (w_F - E + 1) - \big(-(E + w_F + g + g_K)\big) + 1 \\ &= 2w_F + g + g_K + 2. \end{aligned}$$

结论：W 与输入指数 E 无关；E 只决定窗口在常量位串中的起始偏移。

4.2.6 为什么截断不会污染目标位

- 左截断最“接近边界”的被丢位是 $i = w_F - E + 2$，其最低部分积位置为

$$i + E - w_F = 2,$$

已经在 bit1 之上；其进位只会向更高位传播，不会回流到 bit1..bit0 与小数目标位。

- 右截断最“接近边界”的被丢位最高部分积位置低于 bit$-(w_F + g + g_K)$，不会影响保留小数位。

4.3 窗口乘积字段与提取（Q 方案）

从可信窗口乘积中提取：

- $q = k \bmod 4$（2 位，取整数 bit1..bit0）
- y 的定点表示（小数 $w_F + g + g_K$ 位）

由于采用 floor，y 直接是小数部分（无符号，范围 $[0, 1)$），后级直接以同一个 y 驱动双输出函数核。

4.4 与 Octant（$4/\pi$ + round）算法的关键对比

项目	Quadrant（本文 Q）	Octant（备份文档）
常量	$2/\pi$	$4/\pi$
取整	$k = \lfloor p \rfloor$	$k = \mathrm{round}(p)$
周期索引	$q = k \bmod 4$（2 位）	$k \bmod 8$（3 位）
余量区间	$y \in [0, 1)$	$y \in [-\frac{1}{2}, \frac{1}{2}]$
后级结构	双基函数核 + 选择/符号重构（无 pre-map）	常见为 sin/cos 双通路或更复杂重构
Payne–Hanek 有效窗口	$W = 2w_F + g + g_K + 2$	常见推导为 $W = 2w_F + g + g_K + 3$

直观理解：Q 方案只要 2 个象限位，且 floor 不需要“最近整数判定位”，因此窗口需求比 Octant/round 版本少 1 位。

4.5 `2/π` ROM 存储位宽估算

窗口宽度固定为 W，但窗口起始随 E 滑动，因此 ROM 需覆盖“最深”访问。

对二进制浮点，取最大正规无偏置指数

$$E_{\max} = 2^{w_E - 1} - 1,$$

则最低可能访问位约为

$$i_{\min}^{\text{global}} \approx -(E_{\max} + w_F + g + g_K).$$

故 `2/π` 有效位串总深度可按

$$B_{2/\pi} \approx E_{\max} + w_F + g + g_K + 1$$

估算，即

$$B_{2/\pi} \approx 2^{w_E - 1} + w_F + g + g_K.$$

工程上建议再加少量冗余位（例如 +2~+8）用于地址边界与实现简化。

> 备注：与 Octant 常见经验式相比，Q 方案通常可减少常量深度需求；是否显著节省取决于目标格式、是否覆盖次正规输入、以及你选取的 g_K。

---

5. 双输出函数核：同时计算 $s(y)$ 与 $c(y)$

5.1 无 pre-map 的直接输入

函数核直接使用归约输出 $y \in [0,1)$，不再生成 $1-y$。

因此可删除 `pre-map` 级（减法器 + MUX），并减少一级控制与对齐。

5.2 HOTBM 目标函数

在同一输入 y 下并行（或共享部分子结构后同拍）得到：

$$s(y) = \sin\left(\frac{\pi}{2}y\right), \qquad c(y) = \cos\left(\frac{\pi}{2}y\right).$$

可采用两条常见数值稳定分解：

$$s(y) = y \cdot h_s(y), \qquad h_s(y) = \frac{\sin(\frac{\pi}{2}y)}{y},$$

$$c(y) = 1 - h_c(y), \qquad h_c(y) = 1 - \cos\left(\frac{\pi}{2}y\right).$$

其中 h_s, h_c 由 HOTBM 做定点近似，再经乘法/补偿恢复为浮点结果。

5.3 特殊点处理

$y = 0$ 时：

$$s(0) = 0, \qquad c(0) = 1.$$

硬件可对该点走快速旁路，避免不必要乘法。

---

6. 重构规则（mode / 象限 / 符号）

先对 $u = |x|$ 建表，再处理原始符号 S_x。

6.1 对 $u = |x|$ 的重构

记核心输出为 $s(y), c(y)$。

$q = k \bmod 4$	$\sin(u)$	$\cos(u)$
0	$+s(y)$	$+c(y)$
1	$+c(y)$	$-s(y)$
2	$-s(y)$	$-c(y)$
3	$-c(y)$	$+s(y)$

6.2 恢复 x 的符号

利用奇偶性：

$$\sin(-x) = -\sin(x), \qquad \cos(-x) = \cos(x).$$

因此：

- `mode=SIN`：最终符号再与 S_x 异或
- `mode=COS`：最终符号不受 S_x 影响

---

7. 异常与边界处理

- $x = \pm 0$：
  - `SIN` 输出 ± 0（保留符号）
  - `COS` 输出 $+1$
- $x = \pm\infty$：输出 NaN
- $x = \mathrm{NaN}$：输出 NaN

边界一致性：

- $y = 0$（即 p 为整数）时，直接落在 $s(0) = 0, c(0) = 1$，映射仍连续
- 象限边界（q 切换点）由 floor + frac 定义唯一，无重叠

---

8. 误差与资源结论

8.1 误差来源

来源	控制手段
Payne–Hanek 截断	$g_K + g$ 保护位
HOTBM 逼近	minimax + 目标 ulp 约束
中间舍入	级间保护位 g
最终舍入	标准 faithful rounding

8.2 资源特性（相对双通路）

在本项目约束“每次只输出一个函数”下：

- 保留单条流水线
- 移除 pre-map（`1-y` 减法 + MUX）
- 采用同输入双基函数核，末级按 `mode/q` 选择单输出

这与改进后的方案 Q 目标一致：**用适度表面积换取更短关键路径与更直接控制。**

---

9. 设计落地要点（给实现同学）

1. 归约阶段统一对 $u = |x|$ 处理，避免负数 floor 的边界复杂性。
2. 保留 q（2 位）、y（含保护位）以及 S_x。
3. 直接以 y 输入双输出 HOTBM，同时得到 $s(y), c(y)$，再统一做选择/符号重构。

4. 若时序紧张，优先给 HOTBM 输入与输出选择级打拍。
5. 异常路径旁路输出，避免污染主流水线。

以上即方案 Q 的完整文档版定义，可直接作为后续 RTL 与验证规范的上位依据。

---

10. FP16（binary16）输入下的逐步实现细化

本节将方案 Q 落到 **IEEE-754 binary16**（FP16）输入，目标是给 RTL 同学可直接对照实现的“逐拍规格”。

10.1 FP16 参数与内部建议位宽

FP16 基本参数：

- $w_E = 5$
- $w_F = 10$
- $E_0 = 15$

因此（对正规数）无偏置指数范围为 $[-14, 15]$。

为 faithful rounding，建议内部保护位初值：

- 级间保护位：$g = 2$
- Kahan 保护位：$g_K \in [10, 12]$（建议先取 12，再通过穷举回归确认是否可降到 10 或 11）

按本方案窗口公式：

$$W = 2w_F + g + g_K + 2 = 24 + g_K.$$

若 $g_K = 12$，则 $W = 36$。

10.2 Step 0：输入解包与异常分类

输入字段（16 位）：

- `sign_x`：1 位

- `exp_x`：5 位
- `frac_x`：10 位

分类信号：

- `is_zero = (exp_x==0) && (frac_x==0)`
- `is_inf  = (exp_x==31) && (frac_x==0)`
- `is_nan  = (exp_x==31) && (frac_x!=0)`
- `is_sub  = (exp_x==0) && (frac_x!=0)`

建议：主流水线只处理“有限值（含次正规）”，`0/inf/nan` 走异常旁路并在末级与主路仲裁。

10.3 Step 1：归一化到统一尾数形式（含次正规）

统一生成：

- `u_sign = 0`（先取绝对值）
- `mx`：归一化尾数，定点表示 $1.F$（宽度建议 11 位）
- `ex`：无偏置指数（有符号）

实现规则：

1. 正规数：
    - `mx = {1'b1, frac_x}`
    - `ex = exp_x - 15`
2. 次正规数：
    - 对 `frac_x` 做 LZC，左移到 `1.xxxx` 形态
    - `ex = -14 - lzc_shift`

这样后续 Payne–Hanek 统一按

$$u = mx \cdot 2^{ex}$$

处理。

10.4 Step 2：Payne–Hanek 窗口乘法（常量 $2/\pi$）

目标：得到

$$p = u \cdot \frac{2}{\pi} = k + y,$$

并提取：

- $q = k \bmod 4$（2 位）
- y（$w_F + g + g_K$ 位小数，FP16 下为 $12 + g_K$ 位）

实现要点：

1. 预存 `2/pi` 的长位串 ROM（只需 0/1 位常量）
2. 根据 `ex` 计算窗口起始 bit offset
3. 取宽度 W 的窗口 `C`
4. 做定点乘法：`P = mx * C`
5. 从 `P` 的可信区切出：
    - 2 位整数低位给 `q`
    - 后续小数位给 `y_full`

注意：`k` 全值不必完整保留，Q 方案仅需 `k mod 4`。

位宽落地（FP16，建议参数 $g = 2, g_K = 12$）：

- 窗口宽度：$W = 2w_F + g + g_K + 2 = 2 \cdot 10 + 2 + 12 + 2 = 36$
- 最大正规指数：$E_{\max} = 15$
- `2/pi` ROM 估算深度：

$$B_{2/\pi} \approx E_{\max} + w_F + g + g_K + 1 = 15 + 10 + 2 + 12 + 1 = 40 \text{ bit}$$

工程建议：ROM 深度按 44~48 bit 预留，实现上可减少边界处理复杂度。

10.5 Step 3：双输出 HOTBM（输入仅为 y）

函数核以同一个 y 同时生成：

$$s(y) = \sin\left(\frac{\pi}{2}y\right), \qquad c(y) = \cos\left(\frac{\pi}{2}y\right).$$

推荐分解：

1. `h_s(y)` 近似 $\sin(\frac{\pi}{2}y)/y$，并恢复 `s_hat = y * h_s(y)`
2. `h_c(y)` 近似 $1 - \cos(\frac{\pi}{2}y)$，并恢复 `c_hat = 1 - h_c(y)`

3. `s_hat/c_hat` 分别归一化与舍入，打包为 FP16

FP16 精度较低，建议优先选：

- 分段数略多、每段低阶（减小乘法深度）
- 在核末端保留至少 `w_F+3` 位再舍入到 10 位尾数

10.6 Step 4：幅值选择与符号重构

先确定对 $u = |x|$ 的符号：

`mode`	`q`	幅值选择（对 u）	符号（对 u）
SIN	0	$s(y)$	+
SIN	1	$c(y)$	+
SIN	2	$s(y)$	-
SIN	3	$c(y)$	-
COS	0	$c(y)$	+
COS	1	$s(y)$	-
COS	2	$c(y)$	-
COS	3	$s(y)$	+

再恢复输入符号：

- `mode=SIN`：最终符号 `sign_out = sign_u XOR sign_x`
- `mode=COS`：最终符号 `sign_out = sign_u`

其中 `sign_u` 可由 `q` 与 `mode` 的组合逻辑给出。

10.7 Step 5：异常/边界旁路与合并

输出优先级建议：

1. `is_nan` 或 `is_inf`：输出 qNaN
2. `is_zero`：

    - SIN -> `±0`（继承 `sign_x`）
    - COS -> `+1.0`
3. 其余：输出主流水线结果

补充边界：当 `y==0` 时，核输出应满足 `s=0, c=1`，
可在核前/核后加快速检测绕开部分乘法以减小时延与功耗。

10.8 推荐流水切分（FP16 目标高频）

可参考 6 级：

- **S0**：解包 + 分类 + 次正规归一化
- **S1**：`2/pi` 窗口读取 + 乘法前对齐
- **S2**：窗口乘法 + `q/y` 提取
- **S3**：HOTBM 求 `h_s(y), h_c(y)`
- **S4**：恢复 `s(y),c(y)` + 归一化舍入
- **S5**：按 `mode/q` 选择与符号/异常合并

若频率压力大，S4 可再拆成 “查表级 + 组合级”。

10.9 RTL 交付前的最小验证集（FP16）

建议至少覆盖以下向量集：

1. 特殊值：`±0`，`±inf`，`qNaN`，`sNaN`
2. 象限边界：使 p 接近整数（`y≈0`）
3. `y≈1` 场景（验证 $\cos(\frac{\pi}{2}y)$ 近零时的精度与归一化）
4. 次正规输入最小值/最大值
5. 大指数输入（检验 `q` 不误判）
6. 随机 1e6 点对比高精度参考（mpmath/softfloat）

验收标准建议：

- 满足 faithful rounding（误差不超过 1 ulp）
- `mode` 切换无毛刺（时序对齐）
- 异常路径优先级与 IEEE-754 约定一致